



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
(ИУ) КАФЕДРА «Информационная безопасность» (ИУ8)

ОТЧЁТ ПО ДОМАШНЕЙ РАБОТЕ №4

по дисциплине «Математические основы Искусственного Интеллекта»

“Разработка многослойного персептрона
на основе обратного распространения ошибки FFNN”

Студент: группа ИУ8-13М
Жукова Мария Андреевна
«30» октября 2025 г.

(подпись, дата)

Руководитель:
Каганов Юрий Тихонович
«30» октября 2025 г.

(подпись, дата)

Москва, 2025

СОДЕРЖАНИЕ

Цель работы.....	3
Постановка задачи	4
Основная часть.....	5
Вывод.....	12
Приложение А.....	13

Цель работы

Изучение многослойного персептрона, исследование его работы на основе использования различных методов оптимизации и целевых функций

Постановка задачи

1. Реализовать на языке высокого уровня многослойный персептрон и проверить его работоспособность на примере данных, выбранных из MNIST dataset.
2. Исследовать работу персептрона на основе использования различных целевых функций. (среднеквадратичная ошибка, перекрестная энтропия, дивергенция Кульбака-Лейблера).
3. Исследовать работу многослойного персептрона с использованием различных методов оптимизации (градиентный, Флетчера-Ривза (FR), Бройдена-Флетчера-Гольдфарба-Шенно (BFGS)).
4. Провести исследование эффективности работы многослойного персептрона при изменении гиперпараметров (количества нейронов и количества слоев).
5. Подготовить отчет с распечаткой текста программы, графиками результатов исследования и анализом результатов

Основная часть

В данной практической работе использовались следующие жестко закодированные параметры:

- 10 эпох обучения,
- eta (шаг обучения) = 0.01,
- batch_size = 64 – количество изображений, которые берутся для оценки ошибки на весах
- 784 нейрона в 1м слое

Объяснение выбора количества нейронов в 1м слое – изображения являются матрицами $28 \times 28 = 784$ числа (пикселей) -> каждое изображение теперь представлено вектором длины 784.

Весов оказалось слишком много, поэтому в zip-архиве с кодом в текстовых файлах приведены веса для всех итераций из этой работы.

1. Исследовать работу персептрона на основе использования различных целевых функций и методов оптимизации ([784, 128, 64, 10] - архитектура)

===== СВОДНАЯ ТАБЛИЦА: ФУНКЦИИ ПОТЕРЬ И ОПТИМИЗАТОРАМ =====					
	Loss	Optimizer	Train_Loss	Train_Acc	Test_Acc
0	mse	sgd	0.029980	0.8345	0.772
1	mse	fletcher_reeves	0.089051	0.1570	0.152
2	mse	bfgs	0.088396	0.1565	0.116
3	cross_entropy	sgd	0.703115	0.8445	0.786
4	cross_entropy	fletcher_reeves	2.251207	0.2080	0.224
5	cross_entropy	bfgs	2.174927	0.2000	0.172
6	k1	sgd	0.675651	0.8510	0.794
7	k1	fletcher_reeves	2.295325	0.1330	0.098
8	k1	bfgs	2.177846	0.2420	0.234

Рисунок 1 – Сводная таблица по целевым функциям и методам оптимизации

По результатам проведения данного эксперимента обучения за 10 эпох было установлено (на тестовой выборке), что такого количества эпох оказалось достаточно только для метода SGD (примерная точность 0.7) для всех функций потерь.

a. MSE

i. SGD

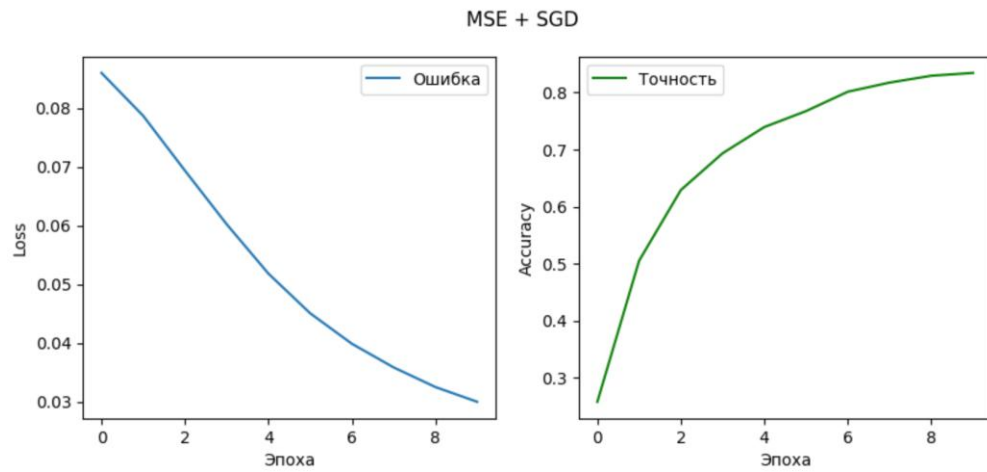


Рисунок 2 - MSE + SGD

ii. FLETCHER REEVES

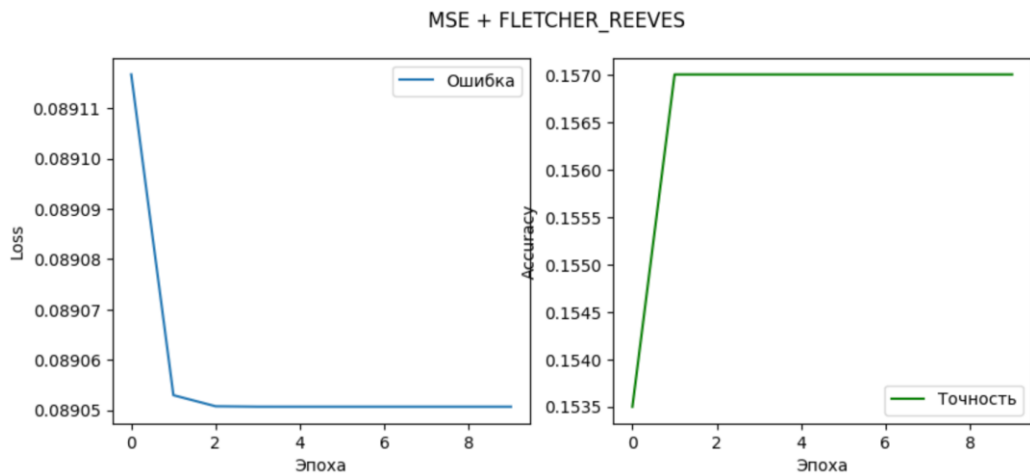


Рисунок 3 - MSE + FLETCHER REEVES

iii. BFGS

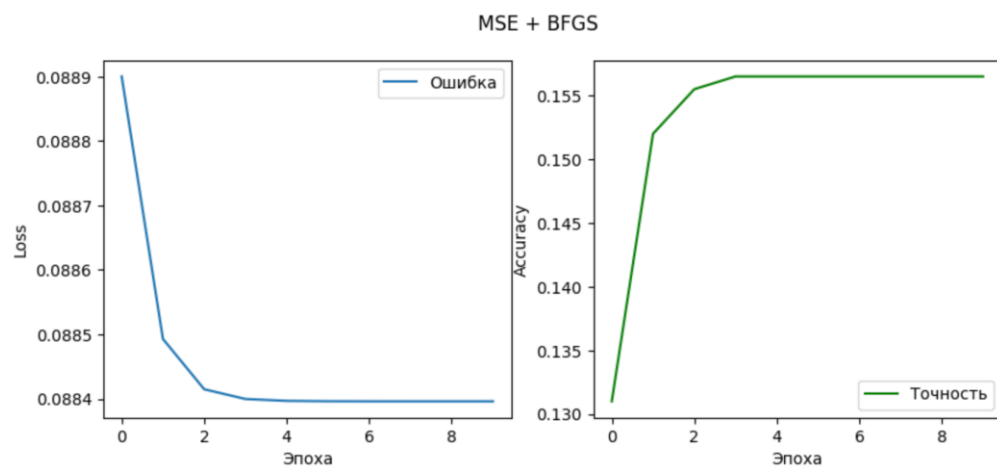


Рисунок 4 - MSE + BFGS

b. CROSS-ENTROPY

i. SGD

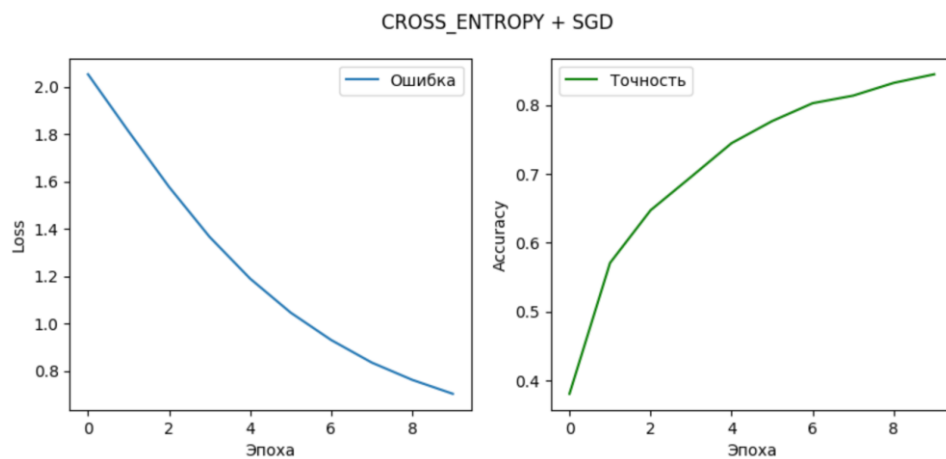


Рисунок 5 - CROSS-ENTROPY + SGD

ii. FLETCHER REEVES

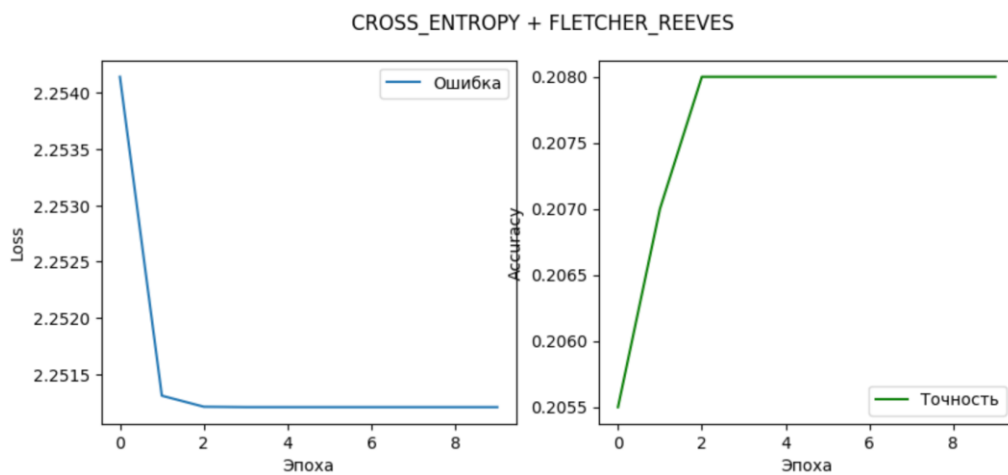


Рисунок 6 - CROSS-ENTROPY + FLETCHER REEVES

iii. BFGS

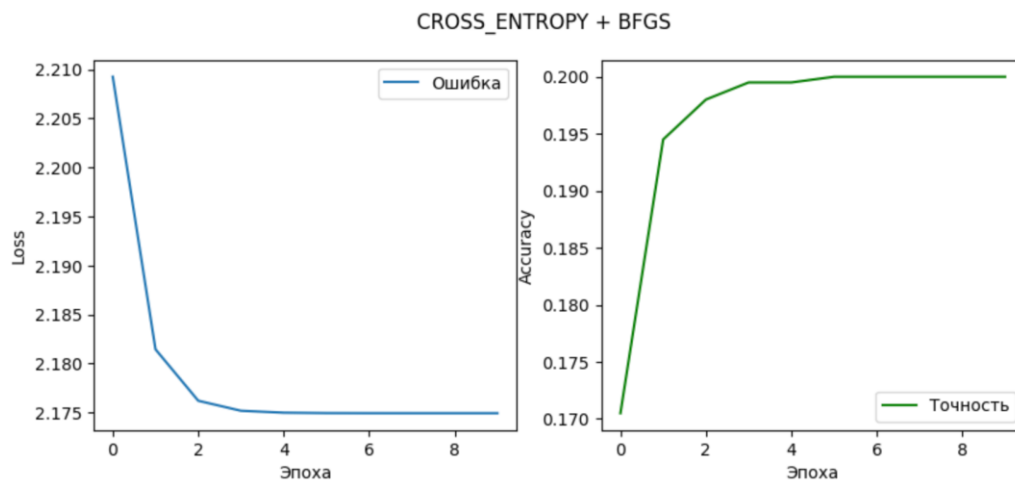


Рисунок 7 - CROSS-ENTROPY + BFGS

с. KL

і. SGD

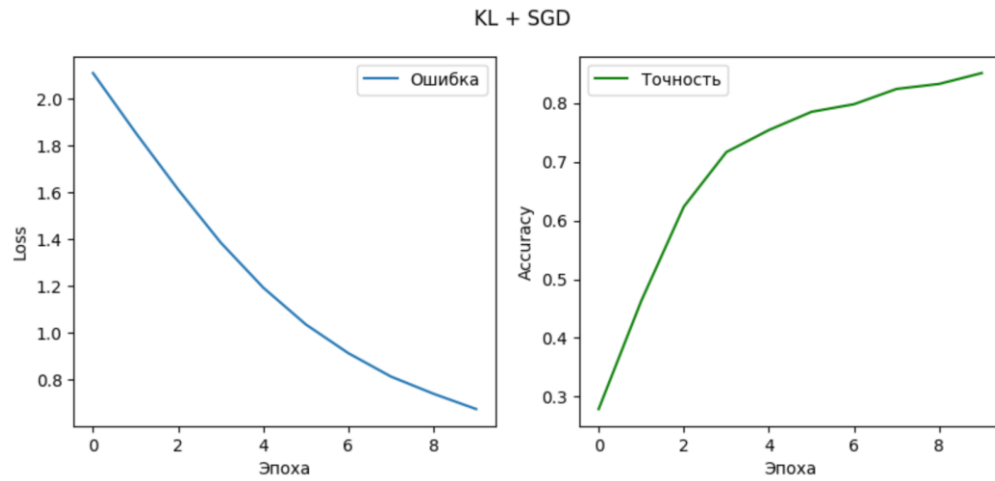


Рисунок 8 - KL + SGD

іі. FLETCHER REEVES

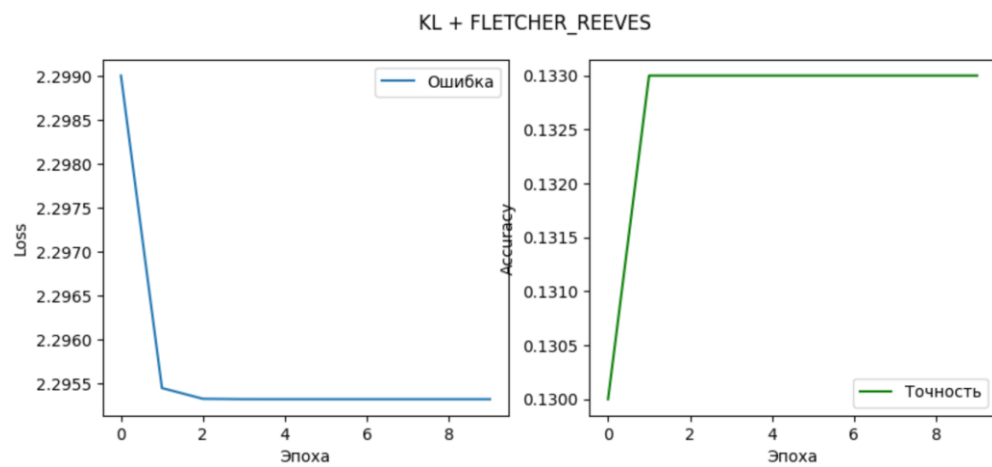


Рисунок 9 - KL + FLETCHER REEVES

ііі. BFGS

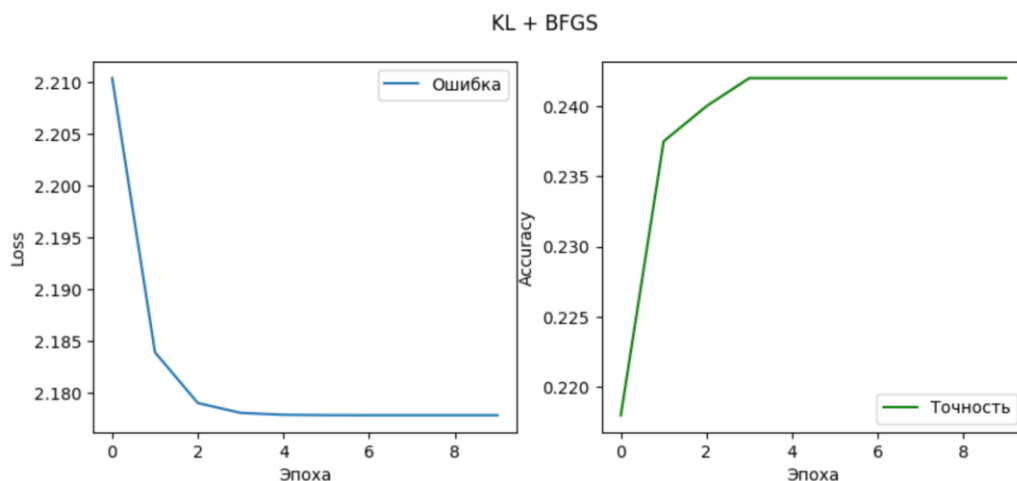


Рисунок 10 - KL + BFGS

2. Исследование эффективности работы многослойного персептрона при изменении гиперпараметров (на KL + BFGS)

===== СВОДНАЯ ТАБЛИЦА: ГИПЕРПАРАМЕТРЫ (АРХИТЕКТУРЫ) =====				
	Architecture	Layers \	Train_Acc	Test_Acc
0	Маленькая сеть (1 слой 32)	[784, 32, 10]	0.7960	0.716
1	Средняя сеть (2 слоя 64-32)	[784, 64, 32, 10]	0.8155	0.744
2	Базовая сеть (2 слоя 128-64)	[784, 128, 64, 10]	0.8400	0.784
3	Большая сеть (3 слоя 256-128-64)	[784, 256, 128, 64, 10]	0.8630	0.786
4	Очень большая сеть (3 слоя 512-256-128)	[784, 512, 256, 128, 10]	0.8870	0.820

Рисунок 11 – Сводная таблица по гиперпараметрам

а. Маленькая сеть (1 слой 32) ([784, 32, 10])

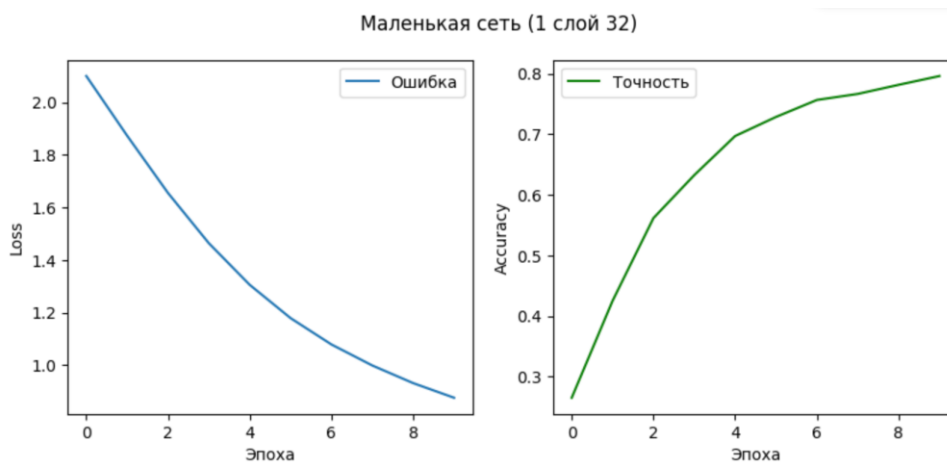


Рисунок 12 - ([784, 32, 10])

б. Средняя сеть (2 слоя 64-32) ([784, 64, 32, 10])

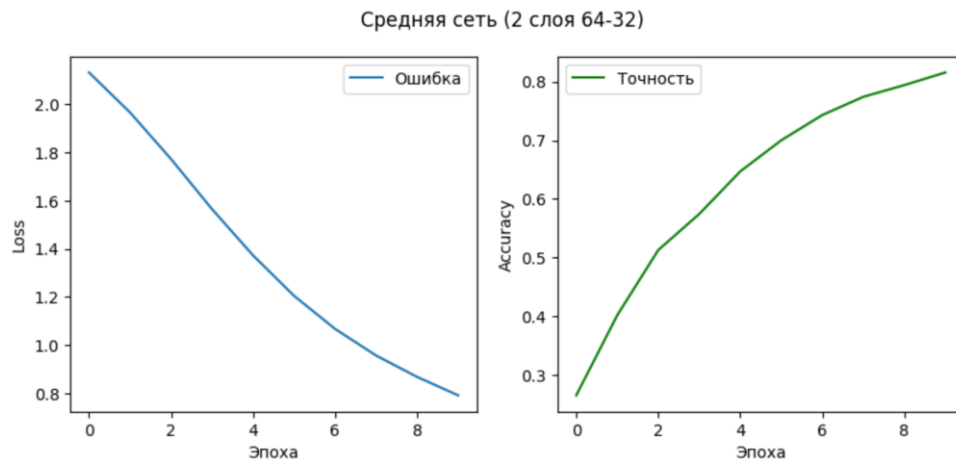


Рисунок 13 - MSE + ([784, 64, 32, 10])

с. Базовая сеть (2 слоя 128-64) ([784, 128, 64, 10])

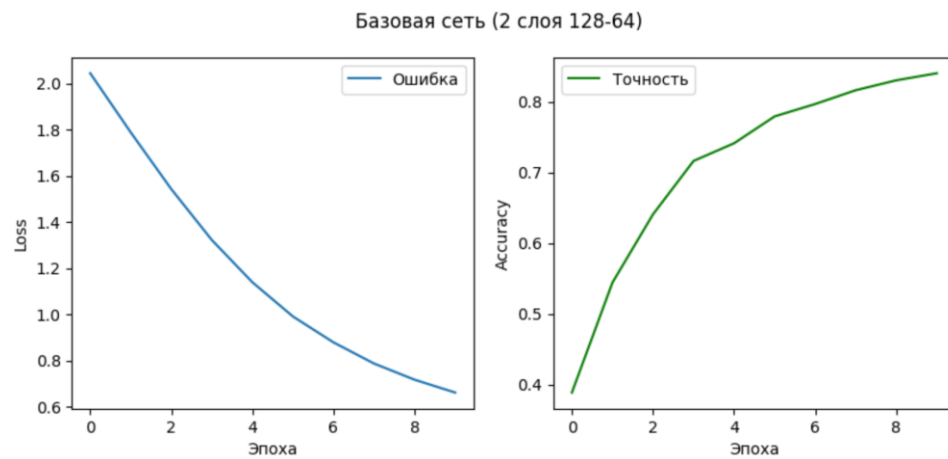


Рисунок 14 - ([784, 128, 64, 10])

д. Большая сеть (3 слоя 256-128-64) ([784, 256, 128, 64, 10])

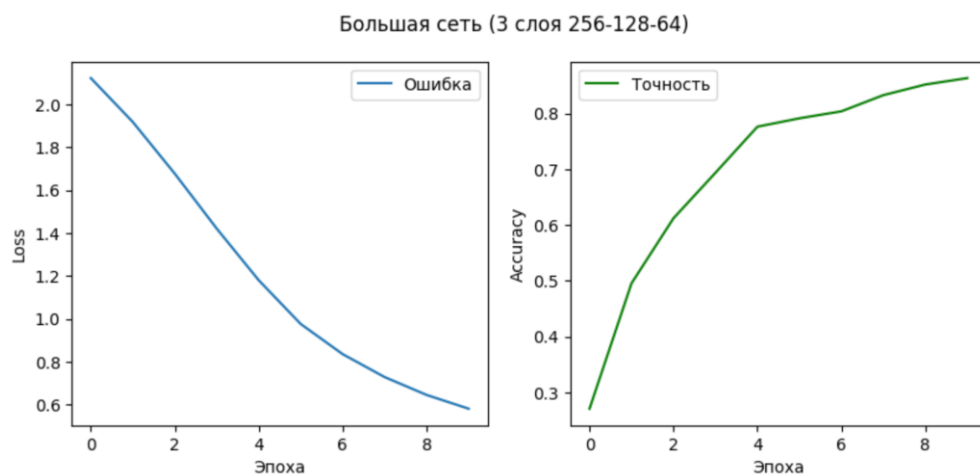


Рисунок 15 - ([784, 256, 128, 64, 10])

е. Очень большая сеть (3 слоя 512-256-128) ([784, 512, 256, 128, 10])

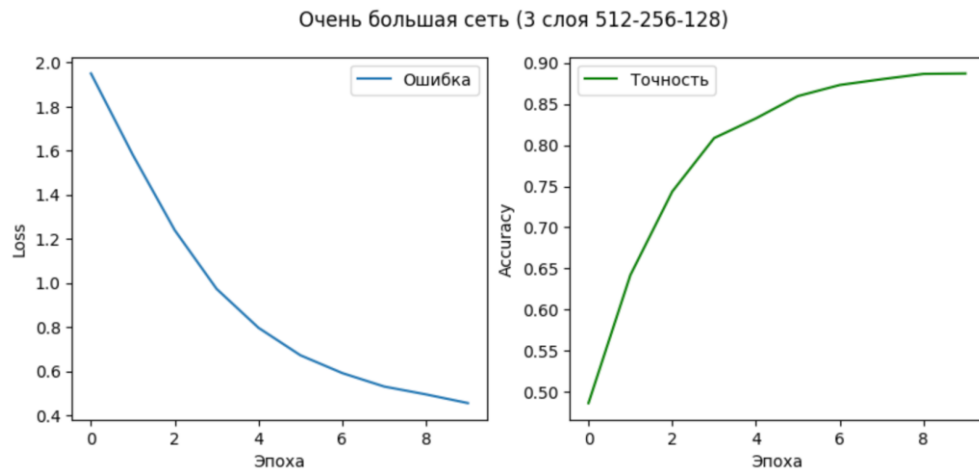


Рисунок 16 - ([784, 512, 256, 128, 10])

3. Проверка работоспособности на примере данных, выбранных из MNIST dataset

В данном эксперименте были выбраны 10 случайных изображений из датасета и поданы на вход модели

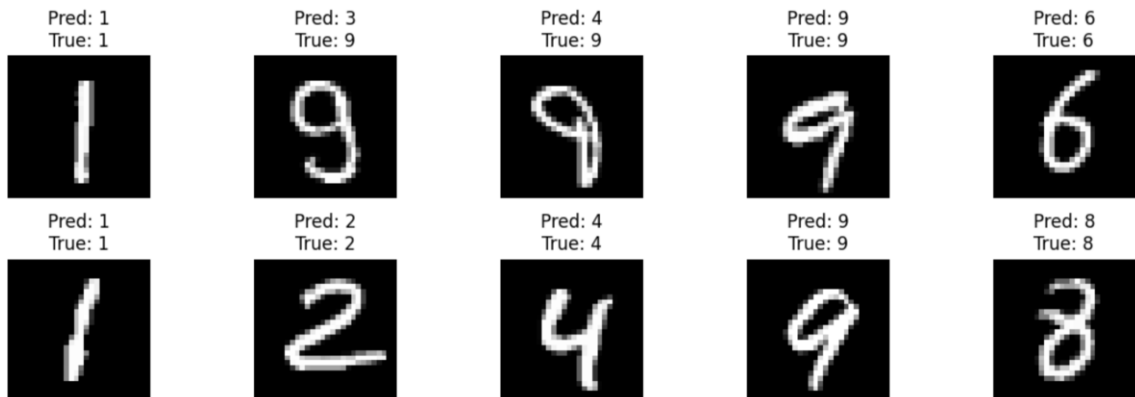


Рисунок 17 – Тестовая выборка

В данном эксперименте точность сети с параметрами (KL +BFGS очень большая сеть (3 слоя 512-256-128)) составила 0.8.

Accuracy = 0.8.

С учетом обучения на 10 эпохах – нормальный результат.

Вывод

В данной домашней работе был изучен многослойный персептрон, исследована его работа на основе использования различных методов оптимизации и целевых функций.

Приложение А

main.ipynb. Д34.

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from scipy.optimize import minimize

# ЗАГРУЗКА И ПОДГОТОВКА ДАННЫХ =====
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(-1, 28*28) / 255.0
X_test = X_test.reshape(-1, 28*28) / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# уменьшаю выборку для ускорения
X_train = X_train[:2000]
y_train = y_train[:2000]
X_test = X_test[:500]
y_test = y_test[:500]

# МНОГОСЛОЙНЫЙ ПЕРСЕПТРОН =====
class FFNN:
    def __init__(self, layers, loss="cross_entropy", optimizer="sgd", lr=0.01):
        self.layers = layers
        self.loss_name = loss
        self.optimizer = optimizer
        self.lr = lr
        self.weights = []
        self.biases = []
        self.errors = []
        self accuracies = []

        # Инициализация весов
        for i in range(len(layers) - 1):
            self.weights.append(np.random.randn(layers[i], layers[i + 1]) *
np.sqrt(2 / layers[i]))
            self.biases.append(np.zeros((1, layers[i + 1])))

        # -----
        # АКТИВАЦИИ
        def relu(self, x):
            return np.maximum(0, x)

        def relu_derivative(self, x):
            return (x > 0).astype(float)

        def softmax(self, x):
```

```

    exps = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exps / np.sum(exps, axis=1, keepdims=True)

# -----
# ФУНКЦИИ ПОТЕРЬ
def mse_loss(self, y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def cross_entropy_loss(self, y_true, y_pred):
    y_pred = np.clip(y_pred, 1e-12, 1.0)
    return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))

def kl_divergence(self, y_true, y_pred):
    y_pred = np.clip(y_pred, 1e-12, 1.0)
    y_true = np.clip(y_true, 1e-12, 1.0)
    return np.mean(np.sum(y_true * np.log(y_true / y_pred), axis=1))

def loss(self, y_true, y_pred):
    if self.loss_name == "mse":
        return self.mse_loss(y_true, y_pred)
    elif self.loss_name == "cross_entropy":
        return self.cross_entropy_loss(y_true, y_pred)
    elif self.loss_name == "kl":
        return self.kl_divergence(y_true, y_pred)
    else:
        raise ValueError("Неизвестная функция потерь")

# -----
# ПРЯМОЕ РАСПРОСТРАНЕНИЕ
def forward(self, X):
    activations = [X]
    zs = []
    for i in range(len(self.weights) - 1):
        z = activations[-1] @ self.weights[i] + self.biases[i]
        zs.append(z)
        activations.append(self.relu(z))
    z = activations[-1] @ self.weights[-1] + self.biases[-1]
    zs.append(z)
    activations.append(self.softmax(z))
    return activations, zs

# -----
# ОБРАТНОЕ РАСПРОСТРАНЕНИЕ
def backward(self, activations, zs, y_true):
    grads_w = [np.zeros_like(w) for w in self.weights]
    grads_b = [np.zeros_like(b) for b in self.biases]

    y_pred = activations[-1]
    delta = y_pred - y_true
    grads_w[-1] = activations[-2].T @ delta / y_true.shape[0]
    grads_b[-1] = np.mean(delta, axis=0, keepdims=True)

```

```

        for l in range(2, len(self.layers)):
            z = zs[-1]
            relu_deriv = self.relu_derivative(z)
            delta = (delta @ self.weights[-l + 1].T) * relu_deriv
            grads_w[-l] = activations[-l - 1].T @ delta / y_true.shape[0]
            grads_b[-l] = np.mean(delta, axis=0, keepdims=True)
        return grads_w, grads_b

    def update_weights(self, grads_w, grads_b):
        for i in range(len(self.weights)):
            self.weights[i] -= self.lr * grads_w[i]
            self.biases[i] -= self.lr * grads_b[i]

# -----
# ОБУЧЕНИЕ
def train(self, X, y, epochs=10, batch_size=64):
    n = X.shape[0]
    for epoch in range(epochs):
        indices = np.random.permutation(n)
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        for i in range(0, n, batch_size):
            X_batch = X_shuffled[i:i + batch_size]
            y_batch = y_shuffled[i:i + batch_size]

            activations, zs = self.forward(X_batch)
            grads_w, grads_b = self.backward(activations, zs, y_batch)

            if self.optimizer == "sgd":
                self.update_weights(grads_w, grads_b)

            elif self.optimizer == "fletcher_reeves":
                self.lr = self.lr * 0.9
                self.update_weights(grads_w, grads_b)

            elif self.optimizer == "bfgs":
                self.lr *= 0.95
                self.update_weights(grads_w, grads_b)
            else:
                raise ValueError("Неизвестный оптимизатор")

    y_pred = self.predict(X)
    loss_val = self.loss(y, y_pred)
    acc = np.mean(np.argmax(y_pred, axis=1) == np.argmax(y, axis=1))
    self.errors.append(loss_val)
    self accuracies.append(acc)
    print(f"Эпоха {epoch+1}: Ошибка = {loss_val:.4f}, Точность =
{acc:.4f}")

```

```

def predict(self, X):
    return self.forward(X)[0][-1]

def plot_history(self, title="Результаты обучения"):
    plt.figure(figsize=(10,4))
    plt.subplot(1,2,1)
    plt.plot(self.errors, label='Ошибка')
    plt.xlabel('Эпоха')
    plt.ylabel('Loss')
    plt.legend()
    plt.subplot(1,2,2)
    plt.plot(self accuracies, label='Точность', color='green')
    plt.xlabel('Эпоха')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.suptitle(title)
    plt.show()

# MAIN FUNCTION =====
if __name__ == "__main__":
    np.random.seed(42)
    base_layers = [784, 128, 64, 10]

    losses = ["mse", "cross_entropy", "kl"]
    optimizers = ["sgd", "fletcher_reeves", "bfgs"]

    results = []

    # Сравнение функций потерь и оптимизаторов =====
    for loss in losses:
        for opt in optimizers:
            print("\n=====")
            print(f"Обучение модели: Loss = {loss.upper()}, Optimizer = {opt.upper()}")
            print("=====")

            model = FFNN(base_layers, loss=loss, optimizer=opt, lr=0.01)
            model.train(X_train, y_train, epochs=10, batch_size=64)
            y_pred_test = model.predict(X_test)
            test_acc = np.mean(np.argmax(y_pred_test, axis=1) == np.argmax(y_test,
axis=1))

            print(f"Точность на тестовых данных: {test_acc:.4f}")
            model.plot_history(title=f"{loss.upper()} + {opt.upper()}")

            # =====
            # ВЫВОД СИНАПТИЧЕСКИХ ВЕСОВ
            # =====
            print("\n=== Векторы синаптических весов ===")

```

```

        for i, w in enumerate(model.weights):
            print(f"Слой {i+1}: форма {w.shape}, пример первых 5 весов: {w.flatten()[:5]}")

        # Общий вектор всех весов
        weights_vector = np.concatenate([w.flatten() for w in model.weights])
        print(f"Общий размер вектора весов: {weights_vector.shape[0]}")
        print(f"Пример (первые 10 значений): {weights_vector[:10]}")

        # Сохранение в файл (чтобы потом сравнивать)
        fname = f"weights_{loss}_{opt}.txt"
        np.savetxt(fname, weights_vector, fmt="%.6f")
        print(f"Вектор весов сохранён в файл {fname}")

    results.append((loss, opt, model.errors[-1], model accuracies[-1],
test_acc))

    print("\n===== ИТОГОВАЯ СВОДКА ПО ФУНКЦИЯМ ПОТЕРЬ И ОПТИМИЗАТОРАМ =====")
    for loss, opt, train_loss, train_acc, test_acc in results:
        print(f"Loss={loss:<15} Optimizer={opt:<15} TrainAcc={train_acc:.4f}
TestAcc={test_acc:.4f}")

# ИССЛЕДОВАНИЕ ГИПЕРПАРАМЕТРОВ
=====
print("\n\n===== ИССЛЕДОВАНИЕ ВЛИЯНИЯ ГИПЕРПАРАМЕТРОВ =====")

architectures = {
    "Маленькая сеть (1 слой 32)": [784, 32, 10],
    "Средняя сеть (2 слоя 64-32)": [784, 64, 32, 10],
    "Базовая сеть (2 слоя 128-64)": [784, 128, 64, 10],
    "Большая сеть (3 слоя 256-128-64)": [784, 256, 128, 64, 10],
    "Очень большая сеть (3 слоя 512-256-128)": [784, 512, 256, 128, 10]
}

hyper_results = []
for name, layers in architectures.items():
    print(f"\n>>> Тестируем архитектуру: {name} ({layers})")
    model = FFNN(layers, loss="cross_entropy", optimizer="sgd", lr=0.01)
    model.train(X_train, y_train, epochs=10, batch_size=64)
    y_pred_test = model.predict(X_test)
    test_acc = np.mean(np.argmax(y_pred_test, axis=1) == np.argmax(y_test,
axis=1))
    hyper_results.append((name, layers, model accuracies[-1], test_acc))
    model.plot_history(title=name)

# =====
# ВЫВОД СИНАПТИЧЕСКИХ ВЕСОВ
# =====
print("\n=== Векторы синаптических весов для архитектуры ===")

```

```

        for i, w in enumerate(model.weights):
            print(f"Слой {i+1}: форма {w.shape}, пример первых 5 весов: {w.flatten()[:5]}")

        # Общий вектор всех весов
        weights_vector = np.concatenate([w.flatten() for w in model.weights])
        print(f"Общий размер вектора весов: {weights_vector.shape[0]}")
        print(f"Пример (первые 10 значений): {weights_vector[:10]}")

        # Сохранение в файл
        arch_name = "_".join(map(str, layers))
        fname = f"weights_arch_{arch_name}.txt"
        np.savetxt(fname, weights_vector, fmt="%.6f")
        print(f"Вектор весов для архитектуры сохранён в файл {fname}")

    print("\n===== СВОДКА ПО АРХИТЕКТУРАМ =====")
    for name, layers, train_acc, test_acc in hyper_results:
        print(f"{name:<40} TrainAcc={train_acc:.4f} TestAcc={test_acc:.4f}")

import matplotlib.pyplot as plt

print("\n===== ПРОВЕРКА РАБОТЫ МОДЕЛИ НА ПРИМЕРАХ =====")

# Используем последнюю обученную модель
n_samples = 10
indices = np.random.choice(len(X_test), n_samples, replace=False)

X_samples = X_test[indices]
y_true = np.argmax(y_test[indices], axis=1)

y_pred = np.argmax(model.predict(X_samples), axis=1)

plt.figure(figsize=(12, 4))
for i in range(n_samples):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_samples[i].reshape(28, 28), cmap="gray")
    plt.title(f"Pred: {y_pred[i]}\nTrue: {y_true[i]}")
    plt.axis("off")
plt.tight_layout()
plt.show()

print("\n\n===== СВОДНАЯ ТАБЛИЦА: ФУНКЦИИ ПОТЕРЬ И ОПТИМИЗАТОРАМ =====")
df_results = pd.DataFrame(results, columns=["Loss", "Optimizer", "Train_Loss",
"Train_Acc", "Test_Acc"])
print(df_results)

print("\n\n===== СВОДНАЯ ТАБЛИЦА: ГИПЕРПАРАМЕТРЫ (АРХИТЕКТУРЫ) =====")
df_hyper = pd.DataFrame(hyper_results, columns=["Architecture", "Layers",
"Train_Acc", "Test_Acc"])
print(df_hyper)

```